Subject: Information Security Lab (ICT 3141) **Date: 03-11-2025**
Semester, Branch, Section: 5th Sem CCE A2
Lab End-Sem Exam: 20 Marks **Duration: 90 min**
Name: **Reg. No:** **Roll No:**

SECURE ELECTRONIC VOTING SYSTEM

Scenario:

A national e-voting system allows citizens to cast votes securely from remote locations. It must provide privacy, verifiability, integrity, and non-repudiation while permitting aggregate computation without revealing individual votes.

Requirements:

Client side: A voter completes an electronic ballot containing a candidate selection and constituency ID. Before submission, the device generates a SHA-512 hash of the ballot file for integrity. It encrypts the ballot with AES-256-CBC and encrypts the AES key with the Election Commission's RSA-4096 public key. For authenticity and non-repudiation, the voter's system signs the SHA-512 hash using an ElGamal digital signature. It also encrypts the numerical candidate ID with Paillier so votes can be tallied without decrypting individual ballots. A trusted timestamp records the submission time and establishes whether the vote was cast before the deadline.

Server side: The server decrypts the AES session key with its RSA private key while retaining the Paillier-encrypted vote values. It verifies the ElGamal signature with the voter's public key, compares the SHA-512 hash to detect tampering, and checks the timestamp against the voting period. It homomorphically aggregates encrypted votes and computes totals without decrypting individual votes. A searchable encryption index supports searches by constituency or time range without exposing vote content. Only the final aggregated ciphertext is decrypted using a threshold decryption scheme, so no single authority can independently view votes.

Task: Write a menu-driven Python program implementing the specified client and server functions. Enforce role-based access control and use cryptographic operations for confidentiality, integrity, and authenticity. Use suitable data structures (such as databases, files, or arrays) to store encrypted, signed, and hashed records securely. Display outputs for all specified functions.

Note: If the Python code is not menu-driven, the total marks obtained will be reduced to its half.

Sample Input–Output:-

Illustrative values: AES key (hex) = 000102030405060708090a0b0c0d0e0f101112131415161718191a1b1c1d1e1f; Paillier n = 3127, g = 3128. Fixed IVs and Paillier randomness are used here only to make the sample reproducible. RSA-4096 key wrapping is abbreviated for print.

```
MAIN MENU: 1 Voter | 2 Election Server | 3 Tally Authority | 4 Exit
Input Choice: 1
VOTER MENU: 1 Cast vote | 2 Back
Input Choice: 1; Ballot ID: B001; Candidate ID: 1; Constituency ID: 101
Ballot UTF-8: CandidateID=1;ConstituencyID=101
Output SHA-512(hex): a2152f5e30dee0d9be483e7ed69a634eaba168e7757fd9d46f8ba7a1236a87125
                     d12b4e6c824a2c7f777dd929e295296854fff55ed4c71b9d84c6b69e0c54ed4
Output IV(hex): 202122232425262728292a2b2c2d2e2f
Output AES-256-CBC CT(hex): 716234bfd0fc5ca9e28d61a9661c5ff97012daf5d446bdc3
                            48bfcd5fe15bc945e4d185a79dd7fff65eb0ffb77f5dd6bc
Output AES key wrapped with RSA-4096: [512-byte ciphertext; abbreviated]
Output ElGamal signature (r, s): (399, 271)
Output Paillier vote CT(hex): 0x734302; Timestamp: 03-11-2025 10:05:00 IST
Output: B001 submitted; searchable constituency/time metadata indexed
Input Choice: 1; Ballot ID: B002; Candidate ID: 1; Constituency ID: 102
Ballot UTF-8: CandidateID=1;ConstituencyID=102
Output SHA-512(hex): 4121092f966fccd24056ffef559c668d96af053bc61a96bd615e12f6ebbfce50
                     6e8e90f25c264c6ee1b72a528e7f4626a1afbde8b65588c48bf41895420c8cfd
Output IV(hex): 303132333435363738393a3b3c3d3e3f
Output AES-256-CBC CT(hex): 3969ec08823f99f130ca97508b5447fa0b253d876a71c6b0
                            0283d185f3b29bcef2655cef3eb4321835c62e3319bd9e20
Output AES key wrapped with RSA-4096: [512-byte ciphertext; abbreviated]
Output ElGamal signature (r, s): (318, 125)
Output Paillier vote CT(hex): 0x7c14a7; Timestamp: 03-11-2025 10:07:00 IST
Output: B002 submitted; searchable constituency/time metadata indexed
Input Choice: 2 -> MAIN MENU; Input Choice: 2
SERVER MENU: 1 Verify ballot | 2 Search index | 3 Back
Input Choice: 1; Ballot ID: B001
Output: RSA private-key unwrap successful; ElGamal signature valid
Output: Recomputed SHA-512 matches stored hash; timestamp within voting period
Output: B001 accepted; Paillier vote remains encrypted
Input Choice: 1; Ballot ID: B002
Output: Signature valid; hash matches; timestamp valid; B002 accepted
Input Choice: 2; Constituency ID: 101 -> Output IDs: B001
Input Choice: 2; Time range: 10:00-10:10 IST -> Output IDs: B001, B002
Input Choice: 3 -> MAIN MENU; Input Choice: 3
```

```
TALLY MENU: 1 Aggregate | 2 Threshold decrypt | 3 Back
Input Choice: 1; Ballot IDs: B001, B002
Output: 0x734302 x 0x7c14a7 mod 3127^2 = 0x10fe4f
Output: Only combined ciphertext submitted for threshold decryption
Input Choice: 2; Threshold authorization: sufficient shares verified
Output: Aggregate candidate-1 count = 2; individual votes not decrypted
Input Choice: 3 -> MAIN MENU; Input Choice: 4 -> Exit
```

Evaluation Scheme:

Writeup (5)	Python functionalities for ballot entry, role menus, and access control		☐
	Python functionalities for AES-256-CBC and RSA-4096 key encryption		☐
	Python functionalities for SHA-512 and ElGamal signatures		☐
	Python functionalities for Paillier tallying and threshold decryption		☐
	Python functionalities for timestamps, search, and secure storage		☐
Execution (15)	Client	Enter candidate selection and constituency ID in a ballot	☐
	Client	Compute a SHA-512 hash of the ballot file	☐
	Client	Encrypt the ballot with AES-256-CBC	☐
	Client	Encrypt the AES key using the RSA-4096 public key	☐
	Client	Create an ElGamal digital signature over the SHA-512 hash	☐
	Client	Encrypt the numerical candidate ID using Paillier	☐
	Client	Attach a trusted submission timestamp	☐
	Server	Decrypt the AES key while retaining encrypted vote values	☐
	Server	Verify the signature and compare the ballot hash	☐
	Server	Check the timestamp against the valid voting period	☐
	Server	Aggregate encrypted votes homomorphically	☐
	Server	Search by constituency or time range without revealing votes	☐
	Server	Decrypt only the aggregate using threshold decryption	☐
	All roles	Enforce role-based menus and protect stored records	☐
	All roles	Display the specified outputs in a complete run	☐

Remarks: